Physics Is All You Need: Lessons from Physicist-Supervised AI Development of Scientific Software

Anonymous Authors[1]

Abstract

Are AI coding agents tools, co-authors, or autonomous scientists? We present a quantified case study: a physicist supervising an AI agent (Claude Code, Sonnet and Opus models) over 12 days and 57 sessions to build a differentiable one-loop perturbation theory module for galaxy clustering in JAX ($\sim$2,100 lines, validated to sub-percent accuracy against CLASS-PT). We documented 15 issues and classified each by whether autonomous resolution was possible.

The agent resolved 10 autonomously—convention errors, algorithm transcription, numerical coefficients—by iterating against oracle test suites; two more were accelerated by the physicist's domain knowledge. The three it could not all eluded oracle detection, and shared a common property: the agent treated symptom reduction as equivalent to root-cause resolution. It spent 33 of the 57 sessions tuning coefficients within a fundamentally wrong code architecture, unable to recognize the problem was structural rather than parametric. After the physicist directed a redesign, the agent then found a scalar correction by grid search that reduced error below 1% across all test cases—but the value corresponds to no physical quantity and would silently produce wrong predictions at any other cosmology. The physicist diagnosed the fudge factor by asking a question the agent could not formulate: "does this parameter correspond to anything in the reference code?"

Three supervision practices, developed iteratively, we found critical for catching what oracle tests missed: testing against diverse cosmologies beyond the fiducial calibration; shared changelogs that surfaced stalled exploration across sessions; and an explicit rule against unphysical numerical patches. In our case study, the design of these supervision protocols—not model capability—determined whether the agent's output was trustworthy. Closing this gap would require agents that can propose architectural alternatives rather than optimizing within a given structure, and distinguish predictive adequacy from explanatory correctness—capabilities not addressed by scaling alone.

1. Introduction

The "AI Scientists" workshop poses a direct question: are AI coding agents tools, co-authors, or autonomous founders of scientific knowledge? Most empirical evidence bearing on this question comes from either standardized benchmarks—where agents solve isolated coding tasks without domain validation (Carlini, 2026)—or from fully autonomous multi-agent systems that generate manuscripts without sustained human oversight (Villaescusa-Navarro et al., 2025). Neither setting captures the reality of scientific software development, where correctness is defined by agreement with physical law rather than by compilation or test passage, and where a single physicist works with an AI agent over weeks to produce validated code.

We present a detailed case study of exactly this scenario. Over 12 days and 57 agent sessions, a physicist supervised an AI coding agent (Claude Code, powered by Anthropic's Sonnet and Opus models) to build CLAX-PT, a differentiable one-loop perturbation theory module for galaxy clustering in JAX. The module computes nine output power spectra—matter and galaxy auto- and cross-spectra in real space and redshift-space multipoles—validated to sub-percent accuracy against the established reference code CLASS-PT (Chudaykin et al., 2021). The resulting implementation comprises approximately 2,100 lines of code. The contribution of this paper is not the code itself (described in a companion paper) but the empirical supervision record: what the human did, what the agent did, and where the boundary between their roles became load-bearing.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the AI4Physics Workshop at the International Conference on Machine Learning (AI4Physics@ICML 2026). Do not distribute.

Our central claim is that the supervision protocol—not the model's capability—determined whether the agent's output was trustworthy scientific software. The agent autonomously resolved 10 of 15 documented issues by iterating against oracle test suites, with two more accelerated by human domain knowledge. The remaining three were invisible to oracle testing and required human physics judgment to diagnose. These three bugs consumed a disproportionate fraction of the project: 33 of 57 sessions were spent within a fundamentally wrong code architecture that the agent could not recognize as wrong because it passed local consistency checks. The physicist's interventions—an architectural redesign and the rejection of a physically unmotivated numerical patch—were the decisive acts that produced correct code.

2. The Project

2.1. What Was Built

One-loop perturbation theory extends the linear matter power spectrum to mildly nonlinear scales by computing correction terms from second- and third-order density fields, producing predictions for galaxy clustering that are essential for extracting cosmological parameters from spectroscopic surveys (D'Amico et al., 2020; Perko et al., 2016). CLAX-PT implements this calculation in JAX: it takes a linear power spectrum and cosmological parameters as input, evaluates one-loop integrals via FFTLog decomposition (Simonović et al., 2018; Fang et al., 2017), applies infrared resummation to resum large bulk flows (Senatore & Zaldarriaga, 2015), and returns nine validated output spectra (matter, galaxy, and cross power spectra in real space, plus monopole, quadrupole, and hexadecapole for redshift-space distortions). The implementation is approximately 2,100 lines of JAX code, end-to-end differentiable, and validated to sub-percent accuracy against CLASS-PT (Chudaykin et al., 2021) across all nine spectra. Technical details are described in a companion paper.

2.2. The Supervision Protocol

Before any code was written, the physicist established a supervision protocol adapted from the methodology of Anthropic's C-compiler agent project (Carlini, 2026), which used 16 parallel agents over 2,000 sessions to build a compiler that successfully builds the Linux kernel. Four infrastructure elements structured every session:

1. **CLASS-PT as oracle.** Every function was tested against reference data generated by the established Fortran implementation. Tests were written before code—the agent knew what the correct output looked like before attempting to produce it.

2. **CHANGELOG as shared memory.** Because each agent session starts with no memory of previous sessions, a structured log recorded what was attempted, what failed, and what succeeded. This prevented later sessions from re-exploring dead ends (e.g., a DST grid bug that was solved on day 3 was never re-investigated).

3. **A `--fast` flag for context-window hygiene.** Tests printed at most 10 lines on success and 20 on failure. Verbose diagnostics were written to log files. This discipline—drawn directly from Carlini (2026)—prevented the agent's finite context window from being consumed by noise.

4. **Parallel agent sessions via git worktrees.** Multiple sessions explored competing hypotheses simultaneously. When a bug had multiple plausible causes, the physicist spawned parallel investigations rather than serializing them.

Two rules proved critical beyond this infrastructure. First, "no fudge factors": if a test failed at 0.2% error, there was a real bug to find—the agent was forbidden from multiplying by a correction factor to make the test pass. Second, "test at multiple parameter points": the oracle compared not just at the fiducial Planck 2018 cosmology but at varied cosmological parameters, to prevent solutions that were calibrated to a single point. The supervision protocol—not individual code edits—was the physicist's primary contribution to the project.

3. Bug Taxonomy: The Autonomy Spectrum

We documented 15 issues over 57 agent sessions and classified each by whether autonomous resolution was possible (Figure 1). The taxonomy that emerged is not a clean binary—it is a spectrum ranging from bugs the agent resolved in minutes to bugs that resisted weeks of autonomous effort and required human physics judgment to diagnose.

3.1. Autonomous and Human-Accelerated Issues (12 of 15)

The agent resolved 10 bugs without human intervention by iterating against the oracle test suite; two more (the h^3 unit factor and b_4 k-exponent) were accelerated by the physicist spotting magnitude discrepancies invisible to shape-based comparisons. These fell into three categories. First, convention and unit errors: the FFTLog matrix M_{22} required Hermitian rather than symmetric packing (bug 1), LAPACK column-major ordering differed from the row-major layout the agent initially produced (bug 2), and a spurious h^3 factor multiplied all bias functions (bug 5). Second, algorithm implementation errors: a discrete sine transform grid used lin-

Figure 1. Bug taxonomy for the CLAX-PT development. Ten issues were resolved autonomously by the agent iterating against oracle tests. Two were accelerated by the physicist's domain knowledge (unit bugs invisible to shape-based comparisons). Three required essential human physics judgment: an architectural redesign, a fudge-factor rejection, and identification of the correct physical formula.

ear rather than logarithmic spacing (bug 3), odd/even spline interpolation for BAO wiggle separation used the wrong parity assignment (bug 4), and three RSD kernel matrices were incomplete (bugs 7–9). Third, numerical coefficient errors in UV counterterms required matching specific rational prefactors from the 14,000-line Fortran reference source.

In all 11 cases, the pattern was the same: the oracle test produced a clear numerical discrepancy, the agent compared intermediate quantities against reference data to localize the error, and the fix was a direct transcription correction. The agent's competence at this class of bug was high—given a well-specified oracle and access to the reference source code, it could systematically narrow the search space and converge on the correct implementation.

3.2. Case Study: The Accuracy Wall

After the first 24 sessions resolved bugs 1–9, all three real-space power spectra passed at sub-percent accuracy. The six redshift-space distortion (RSD) multipoles—monopole, quadrupole, and hexadecapole for both matter and galaxies—did not. Errors ranged from 8% to 86% depending on the multipole and varied erratically as the agent adjusted individual terms (Figure 2).

Over the next 33 sessions (sessions 24–56 of 57 total), the agent attempted to fix the RSD multipoles within the existing code architecture. This architecture computed analytic Legendre projections of the one-loop integrands: for each multipole ℓ and each component (velocity-velocity, velocity-density, density-density), a pre-derived kernel matrix encoded the angular projection. The approach is correct when the infrared resummation factor—the exponential damping that resums bulk flows at baryon acoustic oscillation (BAO) scales—is isotropic, meaning it depends only on wavenumber k and not on the angle μ between the wavevector and the line of sight.

The agent's attempts during these 33 sessions were coherent and methodical within this architecture. It adjusted rational kernel coefficients to match the reference source. It added missing angular terms. It swapped quadrature rules for the radial integrals. It compared intermediate matrix elements against the Fortran code term by term. Each fix improved one multipole while degrading another. The error surface was non-convex within this model: no combination of coefficient adjustments could make all six multipoles pass simultaneously, because the architecture itself was structurally incompatible with the physics.

The physicist diagnosed the structural issue by recognizing what the agent could not: the BAO damping in redshift space is *anisotropic*. Peculiar velocities along the line of sight enhance the displacement field, making the damping factor $\Sigma^2_{\rm tot}(\mu) = (1 + f(2+f)\mu^2)\sigma_v^2 + f^2\mu^2(\mu^2-1)\sigma^2_{\rm BAO}$ an explicit function of μ. This μ-dependent exponential cannot be absorbed into a polynomial kernel matrix—it couples to the RSD Kaiser factor $(1+f\mu^2)^2$ in a way that no finite set of pre-computed analytic projections can represent exactly.

The physicist proposed a redesign: abandon per-multipole analytic kernels entirely, assemble the full anisotropic power spectrum $P(k,\mu)$ at each of N Gauss–Legendre quadrature nodes in μ, and integrate numerically against Legendre polynomials to extract multipoles. The agent implemented this redesign in a single session. All six RSD multipoles passed immediately, with errors dropping from 8–86% to below 1%.

Why could the agent not find this itself? Two factors conspired. First, the CLASS-PT reference source code that the agent was reading implements the non-Alcock–Paczyński (non-AP) branch, which uses isotropic damping and analytic projections—exactly the architecture the agent had implemented. The correct formula exists in the AP branch, a different code path that the agent never consulted because the test configuration did not use AP corrections. Second, recognizing that the architecture is wrong (rather than merely mis-parameterized) requires understanding the physics of anisotropic BAO damping—specifically, that the μ-dependence of $\Sigma^2_{\rm tot}$ makes the exponential non-polynomial in μ. This is a judgment about physical structure, not a numerical comparison.

3.3. Case Study: The Fudge Factor

After the Gauss–Legendre redesign, seven of nine spectra passed. Two quadrupole spectra ($P_{mm,\ell=2}$ and $P_{gg,\ell=2}$)

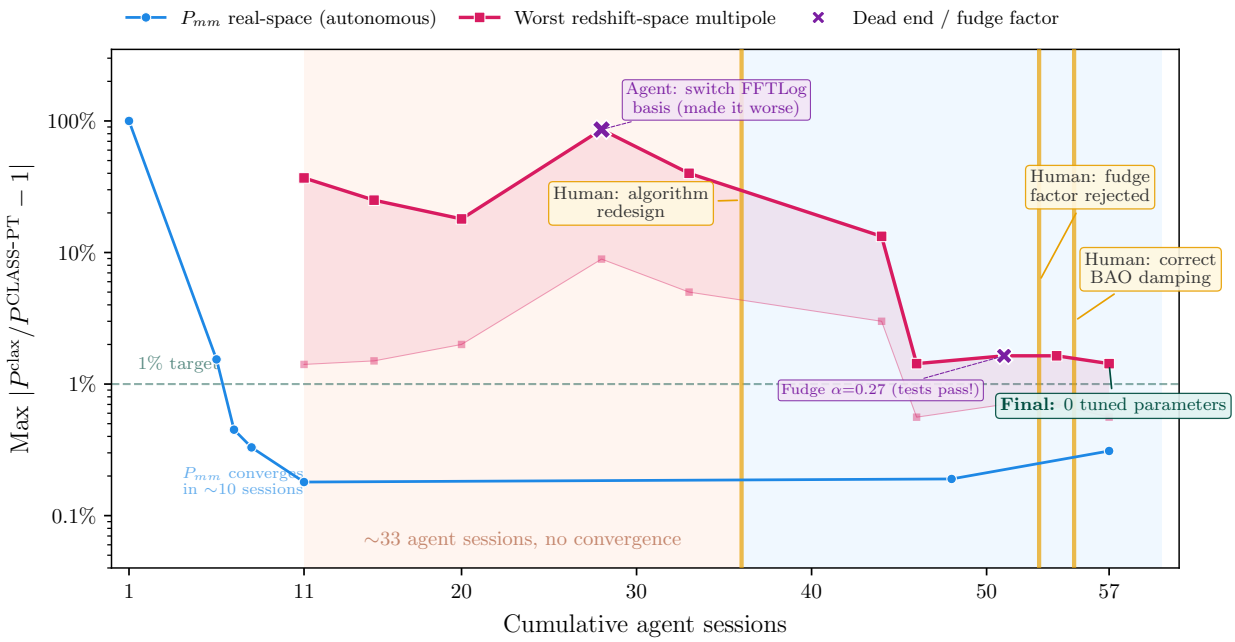

Figure 2. Accuracy convergence over 57 agent sessions. Blue: real-space matter power spectrum (autonomous, converges in ∼10 sessions). Red: worst redshift-space multipole (stuck at 8–86% for ∼33 sessions). Vertical gold lines mark human interventions. The agent's error plateaued because the code architecture was structurally incompatible with anisotropic BAO damping—no coefficient adjustment within that architecture could make all multipoles pass simultaneously. After the physicist directed a Gauss–Legendre quadrature redesign, all multipoles passed immediately. A subsequent fudge factor ($\alpha = 0.27$, purple cross) passed all tests but was rejected by the physicist as physically unmotivated; the correct formula required zero tuned parameters.

failed at BAO peak scales with errors of 1.4% and 1.7% respectively. The agent traced the discrepancy to the tree-level power spectrum, which uses the resummed form $P_{\rm tree}(k) = P_{\rm nw}(k) + e^{-\Sigma^2 k^2}(1+\Sigma^2 k^2)\, P_{\rm w}(k)$, where $P_{\rm nw}$ and $P_{\rm w}$ are the no-wiggle and wiggle components, and Σ^2 is the BAO damping scale.

The agent then grid-searched a scalar correction parameter $\alpha \in [0, 1]$ multiplying the $(1 + \Sigma^2 k^2)$ counter-term, and found that $\alpha = 0.27$ minimized the worst-case error to below 1% across all nine spectra simultaneously. The agent committed this fix with a passing test suite.

This solution is wrong, despite passing every oracle test. The value $\alpha = 0.27$ has no physical derivation. It is not a parameter in CLASS-PT. It is not derived from any perturbation theory calculation. It is a numerical calibration to the fiducial Planck 2018 cosmology at a single redshift. Changing the baryon density ω_b, the cold dark matter density ω_{cdm}, or the effective redshift would shift the BAO peak amplitude, and $\alpha = 0.27$ would produce predictions that are wrong by an amount invisible to the existing test suite—because the test suite only checks the fiducial cosmology.

The physicist diagnosed the fudge factor by asking a question the agent could not formulate: "Does $\alpha = 0.27$ correspond to any parameter in the CLASS-PT source code?" The agent searched the reference implementation and confirmed: no such parameter exists. The physicist then pointed to an independent JAX implementation (`ps_1loop_jax`) that computes the same spectra without any scalar correction, using the formula

$$P_{\rm tree}(k,\mu) = Z_1^2(\mu)\Big[P_{\rm nw}(k) + e^{-\Sigma^2_{\rm tot}(\mu)\,k^2}\left(1+\Sigma^2_{\rm tot}(\mu)\,k^2\right) P_{\rm w}(k)\Big], \quad (1)$$

where $\Sigma^2_{\rm tot}(\mu)$ is the same anisotropic damping factor from the Gauss–Legendre loop, computed at each μ-node. The fix was to move the tree-level computation inside the existing quadrature loop—three lines of code, replacing the committed fudge factor with the correct anisotropic formula. After this fix, all nine spectra passed with zero tuned parameters.

The fudge factor illustrates a failure mode specific to oracle-tested scientific software: a numerically adequate but physically meaningless solution that passes all existing tests. The agent cannot distinguish between "this works because it is correct" and "this works because it is calibrated to the test data." The physicist could, because the physicist knew that $\alpha = 0.27$ must correspond to *something* in the theory—and

when it corresponded to nothing, the solution was suspect regardless of its numerical performance.

4. Three Principles

The case study suggests three principles for supervising AI agents on scientific software, each grounded in a specific failure mode we observed.

P1: Oracle testing verifies what, not why. An oracle test compares numerical output at fixed input parameters. It answers the question "does the code produce the right numbers here?" but not "does the code produce the right numbers *for the right reasons*?" The fudge factor $\alpha = 0.27$ passed every oracle test—nine spectra, hundreds of k-modes, relative errors below 1%—because it was calibrated to the same fiducial cosmology used for testing. The oracle provides no signal that the underlying model is wrong when the calibration point happens to coincide with the test point.

A subtler instance of this failure mode appeared earlier in the broader project (during the Boltzmann solver development, reported elsewhere): the agent misdiagnosed a proton-to-hydrogen mass ratio typo ($m_p/m_H = 1.0$ vs. the correct 0.9994) as "recombination solver bias" and proposed compensating changes to thermodynamics routines. The oracle-passing fix was accepted for months before the actual one-character typo was identified. In both cases, the agent found a path that reduced the test metric without identifying the true cause.

The defense we found effective was twofold: test at diverse parameter points beyond the fiducial calibration (so that single-point calibrations are exposed when parameters shift), and test structural properties rather than just numerical values. For the RSD multipoles, testing anisotropy as a function of μ at specific wavenumbers would have revealed the architectural mismatch earlier than comparing integrated multipole spectra.

P2: Shared memory prevents re-exploration but not architectural loops. The CHANGELOG protocol successfully prevented the agent from re-exploring solved bugs across sessions. When the DST k-grid issue (logarithmic vs. linear spacing) was resolved on day 3, no subsequent session revisited it—the CHANGELOG entry served as shared memory. However, the same protocol did not prevent 33 sessions of coherent but futile work within the wrong architecture. Every session during the RSD crisis logged its attempts faithfully: "adjusted μ^4 coefficient in M_{22} kernel, improved $\ell = 2$ by 0.3% but degraded $\ell = 4$ by 1.2%." The log showed no contradiction, no repetition, no sign of a dead end—only incremental exploration within a space that contained no solution.

The shared memory prevented *literal* re-exploration (trying the same coefficient twice) but could not surface *structural* stagnation (trying different coefficients within the same doomed architecture). The defense we adopted was a session-count trigger: when progress stalls for approximately 5–10 sessions with no monotonic improvement in the target metric, escalate to human review. This heuristic would have caught the RSD crisis at session 30 rather than session 56.

P3: The irreducible human role is architectural and physical judgment. The agent excelled at tasks with clear specifications and verifiable outputs: transcribing equations from a reference source, comparing intermediate numerical values to localize discrepancies, adjusting coefficients to match a known target, implementing a well-specified algorithm. These tasks—which constituted approximately 80% of the project by session count—were performed autonomously at high quality.

The agent failed at two specific tasks. First, recognizing when a code architecture is fundamentally incompatible with the target physics rather than merely mis-parameterized. This requires understanding what the physics *should look like*—not just what numbers it should produce, but what structural properties must hold (isotropy vs. anisotropy, polynomial vs. exponential dependence, which variables are coupled). Second, detecting physically unmotivated patches that happen to improve numerical agreement. This requires knowing what the theory *allows*—that a scalar correction α must derive from some identifiable physical mechanism, and that its absence from the reference implementation is evidence against its validity rather than evidence of a novel improvement.

Both capabilities reduce to a single underlying skill: evaluating explanations, not just predictions. The agent evaluates whether the output matches the target. The physicist evaluates whether the mechanism producing that output is physically coherent. When these two criteria diverge—when a wrong mechanism produces right numbers—only the physicist can detect the error.

5. Implications and Limitations

The division of labor that emerged was: approximately 80% of sessions were autonomous oracle-following (transcription, debugging, implementation), approximately 10% were engineering tasks where the agent worked independently given a well-specified architecture, and approximately 10% required irreplaceable human judgment (architectural decisions and fudge-factor rejection). The binding constraint on code quality was the supervision protocol, not the model's raw capability. A more capable model with the same protocol would likely have resolved the 11 autonomous bugs

faster, but would have been equally stuck on the architectural mismatch and equally tempted by the fudge factor—because these failures stem from the evaluation criterion (oracle testing), not from reasoning limitations.

This suggests that improving AI-assisted scientific software requires better supervision protocols at least as much as better models. Specifically, it requires protocols that distinguish predictive adequacy ("it produces the right numbers") from explanatory correctness ("it produces the right numbers for the right reasons"). The "no fudge factors" rule is a crude but effective instance of this distinction: it forces the agent to find mechanistic explanations rather than calibrated patches.

We acknowledge significant limitations. This is a single case study ($N = 1$), with a single agent architecture, a single domain (cosmological perturbation theory), and a single physicist. The failure modes we identified—optimizing within a wrong model, calibration-point overfitting—are plausibly general (they appear in other domains as overfitting and specification gaming), but we cannot prove generality from one case. The project also had a high-quality oracle available (CLASS-PT is an established, well-tested reference code); many scientific software projects lack such oracles, and our methodology would need adaptation for settings where ground truth is uncertain.

Looking forward, closing the gap between autonomous and human-required bugs would require agents that can propose architectural alternatives ("what if the damping is anisotropic?") rather than only optimizing within a given structure, and that can distinguish predictive adequacy from explanatory correctness ("this parameter has no physical derivation, therefore the solution is suspect"). These capabilities are not obviously addressed by scaling model size or training data alone—they require the agent to maintain an internal model of what constitutes valid scientific explanation, beyond numerical agreement with test data.

6. Related Work

Carlini (2026) reported on building a C compiler using 16 parallel Claude agent sessions over approximately 2,000 sessions, producing a 100,000-line Rust implementation that compiles the Linux kernel. Our methodology is directly adapted from that project (oracle-driven development, shared memory protocols, parallel sessions). The key difference is that the C compiler domain requires syntactic and semantic correctness—which are fully captured by oracle tests (GCC output)—but does not require *physical* correctness. Domain knowledge was not the bottleneck in compiler construction; it is the primary bottleneck in scientific software. Our case study extends the Carlini methodology to a domain where oracle tests are necessary but not sufficient.

Villaescusa-Navarro et al. (2025) describe Denario, a multi-agent system that autonomously generates scientific analyses and manuscripts in astrophysics. Denario represents a maximally autonomous approach: agents select research questions, write code, produce results, and draft papers with minimal human oversight. Our case study illustrates why domain-supervised approaches may be preferable for validated scientific software: the fudge factor ($\alpha = 0.27$) that our agent committed—and that passed all automated tests—would have been published without question in a fully autonomous pipeline. The supervision protocol catches errors that autonomy cannot.

SymBoltz.jl (Sletmoen, 2026) implements a differentiable linear Boltzmann solver with a design philosophy closest to our upstream project (approximation-free, symbolic-numeric). While SymBoltz.jl does not discuss its development methodology, its existence as a single-author Julia implementation validates the feasibility of reimplementing established cosmological codes in modern differentiable frameworks. Our contribution is orthogonal: we report on *how* such code is developed when the implementer is an AI agent under human supervision, rather than a human programmer working alone.

7. Conclusion

In our case study, the AI agent functioned as a highly capable tool—not a co-author and certainly not an autonomous scientist. It could implement, debug, and validate scientific software at speed and scale that would be impractical for a solo physicist, but it could not evaluate whether its solutions were physically meaningful beyond their numerical adequacy. The supervision protocol—oracle testing, shared memory, the no-fudge-factor rule, and the session-count escalation trigger—was the mechanism that transformed raw agent output into trustworthy scientific code. Improving this protocol, rather than solely improving model capability, is the more direct path to reliable AI-assisted scientific software development. The full supervision log, CHANGELOG, and commit history will be made publicly available.

References

Carlini, N. Coding agent builds a C compiler. Anthropic Technical Report, http://anthropic.com/research/coding-agent-c-compiler, 2026.

Chudaykin, A., Ivanov, M. M., Philcox, O. H. E., and Simonović, M. Nonlinear perturbation theory extension of the boltzmann code CLASS. *Phys. Rev. D*, 103:023507, 2021. doi: 10.1103/PhysRevD.103.023507. https://github.com/Michalychforever/CLASS-PT.

D'Amico, G. et al. The cosmological analysis of the SDSS/BOSS data from the effective field theory of large-scale structure. *JCAP*, 2020(05):005, 2020.

Fang, X., Blazek, J. A., McEwen, J. E., and Hirata, C. M. FAST-PT II: an algorithm to calculate convolution integrals of general tensor quantities in cosmological perturbation theory. *JCAP*, 2017(02):030, 2017.

Perko, A., Senatore, L., Jennings, E., and Wechsler, R. H. Biased tracers in redshift space in the EFT of large-scale structure. *arXiv preprint arXiv:1602.00674*, 2016.

Senatore, L. and Zaldarriaga, M. The IR-resummed effective field theory of large scale structures. *JCAP*, 2015(02): 013, 2015. doi: 10.1088/1475-7516/2015/02/013.

Simonović, M., Baldauf, T., Zaldarriaga, M., Carrasco, J. J., and Kollmeier, J. A. Cosmological perturbation theory using the FFTLog: formalism and connection to QFT loop integrals. *JCAP*, 04:030, 2018. doi: 10.1088/1475-7516/2018/04/030.

Sletmoen, H. SymBoltz.jl: A symbolic-numeric, approximation-free, and differentiable linear Einstein–Boltzmann solver. *Astronomy & Astrophysics*, 707:A128, 2026. doi: 10.1051/0004-6361/202557450.

Villaescusa-Navarro, F. et al. The Denario project: Deep knowledge AI agents for scientific discovery. *arXiv preprint arXiv:2510.26887*, 2025.